



BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.

4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.

2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the actual set of all distinct configurations the environment can be in here, every (x, y) cell not occupied by a wall plus the actions that connect them. It's a fixed, finite graph that exists independently of any particular search run. The search tree is what the algorithm builds while exploring that space: each node represents a path from the start state, not just a state. Because different paths can reach the same state, the same state can appear as multiple nodes in the tree the tree is generally much larger than the state space, and can even be infinite if cycles aren't checked.

2. (Understand) In Step 1.2, you were instructed to use a ``reached`` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It solves the problem of re-expanding states already visited via a different path (redundant paths / cycles). Without it, a Tree Search treats every path as new, so on this grid where you can walk in a loop (e.g., right, right, left, left) the algorithm can revisit the same cell endlessly. For DFS specifically, since it always pushes the newest node and pops it immediately, an unbounded grid with a cycle would send it into an infinite loop, and it would never terminate or find the goal, since it has no memory of where it's already been.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS expands nodes level by level (shallowest depth first), so the first time it reaches the goal, it does so along the shortest possible number of steps since every action costs the same (1 step), shortest-in-steps is optimal. DFS instead commits fully to one branch, diving as deep as possible before backtracking, so it can wander far away from the goal before it happens to stumble onto it the path it returns is whatever path it happened to find first, not the shortest one, which is why it looks winding and erratic in the simulation.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS/UCS: frontier size grows roughly with the number of nodes at the current depth in the worst case, space complexity is $O(b^d)$ (b = branching factor, d = depth), since the entire frontier plus all reached states must be kept in memory simultaneously. On a 1,000,000-cell grid, the frontier can balloon to hold a large fraction of all cells before the goal is found, which is why it can exhaust memory and crash.

DFS: only needs to store the single current path plus the alternatives left at each level space complexity is $O(bm)$ (m = maximum depth), which grows *linearly* rather than exponentially. So DFS is far more memory-safe at scale, even though (as above) it isn't optimal and can take much longer or wander further to reach the goal.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING